微信公众号 AI Content Agent（V1）

一个**真的会用工具**的公众号内容创作助手。

你告诉它「写一篇关于 2026 年 AI Agent 最新发展的公众号文章」， 它会自己判断要不要查资料 → 自己调用搜索工具 → 自己决定打开哪个网页读原文 → 综合资料写出一篇结构完整的文章 → 把用到的来源交给你核对。

它不会自动发布。 V1 只生成草稿，发布这一下永远由你亲自点。

目录

- 一、这个项目能做什么
- 二、它是怎么工作的
- 三、技术栈
- 四、目录结构
- 五、快速开始
- 六、环境变量说明
- 七、数据库表
- 八、接口文档
- 九、两个核心工具
- 十、安全设计
- 十一、跑测试
- 十二、常见问题
- 十三、V1 没做的事

一、这个项目能做什么

能力	说明
自主研究	Agent 自己决定搜索什么、搜几次、读哪个页面，不需要你写流程
真实来源	文章引用的每个链接，都必须是它**本次真的检索到或读过**的，编造的一律删除
结构化产出	标题、摘要、开头钩子、大纲、正文、配图建议，一次给全
过程可见	前端能看到「搜索了什么 → 拿到几个来源 → 读了哪篇」，不是黑盒
风险提示	资料不足、工具失败、达到调用上限，都会明确写出来，而不是假装一切正常
人工编辑	生成的是草稿，你可以改标题、改正文、标记状态

二、它是怎么工作的

主流程

```
用户填需求（主题 / 读者 / 目标 / 风格 / 篇幅）
        ↓
Django 校验参数，创建 AgentRun 记录
        ↓
★ Agent 自己做决定（这一步是循环的，不是固定流程）
        ↓
   大模型看到需求 ──→ 判断信息够不够
        ↓ 不够
   调用 web_search  ──→ 拿到一批搜索结果（自动登记来源）
        ↓
   大模型看搜索结果 ──→ 判断哪几篇值得细读
        ↓
   调用 read_webpage ──→ 读到网页正文（自动登记来源）
        ↓ 够了
   调用结构化输出工具，返回 ArticleDraft
        ↓
系统校验：文章里的来源 URL 真的出现过吗？
        ↓ 没出现过 → 删掉，并记一条风险提示
Markdown 转安全 HTML（防 XSS）
        ↓
写库：Article（草稿）+ ToolCall + Source + Message
        ↓
前端展示，你人工确认
```

关键点：第 4~8 步的循环次数和顺序，是模型自己决定的，不是代码写死的。 代码只负责给它工具和规矩。这就是 Agent 和「固定工作流」的区别。

三道防失控的闸门

AI 自己决定调用工具，就存在「搜起来没完」的风险。所以有三层保护：

闸门	配置项	默认值	超了会怎样
工具调用次数	`MAX_TOOL_CALLS`	8	停止研究，用现有资料把文章写完，并加风险提示
单次搜索结果数	`MAX_SEARCH_RESULTS`	5（硬上限 10）	夹到上限
思考步数	`AGENT_RECURSION_LIMIT`	30	LangGraph 强制中断，返回明确错误

注意：达到上限时不会失败，而是「尽力交付 + 如实告知」。 这比直接报错有用得多。

三、技术栈

后端：Python 3.11+、Django 5.2、Django REST Framework、LangChain 1.x（`create_agent`）、 Pydantic、httpx、BeautifulSoup4、trafilatura、MySQL 8 / SQLite

前端：Vue 3、Vite、TypeScript、Vue Router、Pinia、Axios、Element Plus、markdown-it

> **关于 LangGraph**：它已经作为依赖装好了，V1 不直接用。`create_agent` 本身就是用 LangGraph 实现的，以后要做「大纲人工确认 → 写作 → 事实核查」这种固定流程时，可以平滑换成 LangGraph 的 `StateGraph`，不用重写业务代码。

四、目录结构

```
agent_ai_biaoshu/
├── .gitignore                    # ★ 第一条规则就是忽略 .env
├── README.md                     # 你正在看的这份
│
├── backend/                      # Django 后端
│   ├── manage.py
│   ├── requirements.txt
│   ├── pytest.ini                # 测试配置（强制用内存 SQLite）
│   ├── conftest.py               # 全局测试 fixture
│   ├── .env.example              # ★ 环境变量模板，复制成 .env 再填
│   ├── .env                      # 你的真实密钥（已被 git 忽略，不要提交）
│   │
│   ├── config/                   # 项目配置
│   │   ├── settings.py           # 所有配置都从环境变量读
│   │   ├── settings_test.py      # 测试专用配置
│   │   └── urls.py               # 根路由
│   │
│   └── apps/
│       ├── common/               # 公共设施
│       │   ├── exceptions.py     # 统一业务异常 + 全局异常处理器
│       │   ├── responses.py      # 统一返回信封
│       │   ├── middleware.py     # request_id 中间件
│       │   ├── markdown_renderer.py  # Markdown → 安全 HTML
│       │   ├── pagination.py
│       │   └── utils.py
│       │
│       ├── accounts/             # 登录注册（复用 Django 自带 User）
│       │
│       ├── conversations/        # 会话与消息
│       │   └── models.py         # Conversation / Message
│       │
│       ├── agent/                # ★ Agent 核心
│       │   ├── models.py         # AgentRun / ToolCall / Source
│       │   ├── serializers.py
│       │   ├── views.py          # 接口（很薄，只做校验和转发）
│       │   ├── urls.py
│       │   ├── agents/
│       │   │   ├── content_agent.py    # ★ 用 create_agent 搭 Agent
│       │   │   └── audit_middleware.py # 记录每次工具调用
│       │   ├── services/
│       │   │   ├── model_factory.py    # ★ 全项目唯一读 API Key 的地方
│       │   │   ├── source_registry.py  # ★ 防 AI 编造来源
│       │   │   └── agent_service.py    # ★ 主流程编排
│       │   ├── tools/
│       │   │   ├── url_safety.py       # ★ 防 SSRF
```

```
│       │   │   ├── web_search.py       # 工具 1：联网搜索
│       │   │   └── read_webpage.py     # 工具 2：读网页正文
│       │   ├── prompts/
│       │   │   └── content_agent_prompt.py  # System Prompt
│       │   ├── schemas/
│       │   │   └── article.py        # ArticleDraft 结构化输出定义
│       │   └── tests/                # 138 个测试
│       │
│       └── articles/             # 文章
│           ├── models.py         # Article
│           ├── services.py       # 所有写操作都在这里
│           ├── serializers.py
│           └── views.py
│
└── frontend/                     # Vue 前端
    ├── package.json
    ├── vite.config.ts
    ├── .env.example
    ├── README.md                 # 前端自己的说明
    └── src/
        ├── api/                  # 接口封装（http.ts 负责拆信封 + 错误翻译）
        ├── stores/               # Pinia 状态
        ├── pages/                # 页面
        ├── components/           # 组件
        ├── types/                # TypeScript 类型
        └── utils/
```

五、快速开始

0. 先确认环境

```
python --version     # 需要 3.11 或更高
node -v              # 需要 18 或更高
```

1. 启动后端

```
cd backend

# 创建并激活虚拟环境
python -m venv .venv

# Windows:
.venv\Scripts\activate
# macOS / Linux:
# source .venv/bin/activate

# 安装依赖
pip install -r requirements.txt
```

```
# 复制环境变量模板
# Windows:
copy .env.example .env
# macOS / Linux:
# cp .env.example .env

# ★ 现在打开 .env，把 LLM_API_KEY 和 LLM_MODEL 填上（见第六节）

# 建表
python manage.py migrate

# 创建管理员账号（可以登录 /admin 后台看数据）
python manage.py createsuperuser

# 启动
python manage.py runserver 8000
```

看到 `Starting development server at http://127.0.0.1:8000/` 就成功了。 浏览器打开 http://127.0.0.1:8000/health/，应该看到：

```
{"success": true, "data": {"status": "ok"}, "message": ""}
```

2. 启动前端

另开一个终端（后端要保持运行）：

```
cd frontend

npm install

# 如果后端不是 8000 端口，改一下 .env 里的 VITE_API_BASE_URL
# copy .env.example .env

npm run dev
```

浏览器打开终端里提示的地址（默认 http://localhost:5173）。

3. 生成第一篇文章

1. 打开前端页面，先**注册一个账号**（或直接登录）
2. 点「创建文章」
3. 主题填：`2026 年 AI Agent 最新发展`
4. 其他项可以用默认值，点「开始生成」
5. **耐心等待 30~120 秒** —— 它真的在联网搜索和读网页
6. 生成完会自动跳到文章详情页，你能看到正文、引用来源、风险提示

如果想看 Agent 到底做了什么，回到首页看「最近的运行」， 里面能看到每一次工具调用的参数和结果预览。

六、环境变量说明

所有配置都在 `backend/.env` 里。**这个文件已经被 `.gitignore` 忽略，不会被提交。**

必须填的（不填就跑不起来）

变量	说明	例子
`LLM_API_KEY`	大模型的 API Key	`sk-xxxxxxxx`
`LLM_MODEL`	模型名	`gpt-4o-mini` / `deepseek-chat` / `qwen-plus`

按需填的

变量	说明
`LLM_PROVIDER`	默认 `openai`。只要你的服务兼容 OpenAI 接口就填 `openai`
`LLM_BASE_URL`	用国内兼容服务时填。DeepSeek 填 `https://api.deepseek.com/v1`
`SEARCH_API_KEY`	搜索服务密钥。**留空也能跑**，只是 Agent 拿不到外部资料，会在风险提示里说明
`SEARCH_PROVIDER`	默认 `tavily`，去 https://tavily.com 注册有免费额度
`DB_ENGINE`	`mysql` 或 `sqlite`，见下
`DJANGO_SECRET_KEY`	生产环境必须换成随机字符串

关于数据库

方案	什么时候用	怎么配
SQLite	本地开发、想立刻跑起来	`DB_ENGINE=sqlite`，其余不用管
MySQL	正式使用	`DB_ENGINE=mysql`，然后填 `DB_NAME` / `DB_USER` / `DB_PASSWORD` / `DB_HOST` / `DB_PORT`

> ⚠ **MySQL 版本要求：Django 5.2 需要 MySQL 8.0.11 或更高。** 如果你本机装的是 MySQL 5.7，会因为版本过低无法连接（这是 Django 的硬性要求，不是配置问题）。 两个选择：升级到 MySQL 8，或者先用 `DB_ENGINE=sqlite` 把项目跑起来。
>
> 建库语句：
>
> ```
> CREATE DATABASE ai_content_agent CHARACTER SET utf8mb4 COLLATE
> utf8mb4_unicode_ci;
> ```
>
> 必须是 `utf8mb4`，否则存中文和 emoji 会出问题。

七、数据库表

用 `python manage.py migrate` 自动创建。

`agent_run` —— 一次生成任务

字段	说明
`id`	UUID 主键，对外叫 `run_id`
`conversation_id`	属于哪个会话
`status`	`pending` / `running` / `completed` / `failed` / `cancelled`
`input_text`	实际发给模型的完整输入（方便复盘）
`model_name`	这次用的模型
`tool_call_count`	调用了几次工具
`error_message`	失败时的中文提示（**不含堆栈和密钥**）
`started_at` / `completed_at`	耗时统计

`tool_call` —— 每一次工具调用

字段	说明
`agent_run_id`	属于哪次运行
`tool_name`	`web_search` / `read_webpage`
`arguments`	调用参数（JSON）
`status`	`running` / `success` / `failed`
`result_preview`	**只存摘要，最多 500 字**，不存网页全文
`duration_ms` / `sequence`	耗时、顺序

`source` —— 检索到的来源

字段	说明
`agent_run_id`	属于哪次运行
`url` / `url_hash`	原始地址和它的 sha256（唯一约束用哈希，因为 URL 太长建不了索引）
`domain`	域名
`snippet`	**摘要，最多 500 字**
`source_type`	`search`（搜到的）/ `webpage`（真的读过全文的）
`is_cited` / `usage`	文章有没有引用它、用来支撑什么观点

`article` —— 文章

字段	说明
`title` / `subtitle` / `summary` / `hook` / `outline`	标题、副标题、摘要、开头钩子、大纲
`content_markdown`	正文原始 Markdown
`content_html`	渲染并**清洗过**的 HTML（可以直接粘进公众号后台）
`image_suggestions`	配图建议
`source_ids`	引用的来源 id 列表
`risk_notes`	风险提示（模型给的 + 系统检测到的）
`status`	`draft` / `review` / `approved` / `archived`
`version`	每次编辑 +1
`model_name` / `tool_call_count`	溯源信息：这篇是怎么生成的

`conversation` / `message` —— 会话与消息

记录每一轮对话。`Message.message_type` 区分 `text` / `tool_call` / `tool_result` / `article`。

八、接口文档

统一前缀 `/api/v1`，统一认证方式 `Authorization: Token <你的token>`。

返回格式

成功：

```
{ "success": true, "data": { ... }, "message": "" }
```

失败：

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "请填写文章主题。",
    "request_id": "1c4021fe"
  }
}
```

> 报错时把 `request_id` 给后端，能直接在日志里定位到这一次请求。 **失败响应里永远不会有堆栈、文件路径、API Key。**

接口列表

方法	路径	说明
POST	`/auth/register`	注册，直接返回 token
POST	`/auth/login`	登录，返回 token
POST	`/auth/logout`	退出（删掉 token）
GET	`/auth/me`	当前用户信息
POST	**`/agent/runs`**	**★ 创建生成任务（同步，会等 30~120 秒）**
GET	`/agent/runs`	我的历史运行记录
GET	`/agent/runs/{run_id}`	某次运行的状态
GET	`/agent/runs/{run_id}/tool-calls`	工具调用过程（前端时间线）
GET	`/agent/runs/{run_id}/sources`	这次检索到的所有来源
GET	`/agent/conversations`	我的会话列表
GET	`/agent/conversations/{id}/messages`	会话里的消息
GET	`/articles`	文章列表（支持 `?status= ?keyword= ?page=`）
GET	`/articles/stats/`	首页统计
GET	`/articles/{id}`	文章详情
PATCH	`/articles/{id}`	编辑文章
DELETE	`/articles/{id}`	归档（软删除）
GET	`/articles/{id}/sources`	这篇文章引用的来源

常用错误码

code	HTTP	含义
`VALIDATION_ERROR`	400	参数不对，`message` 里有中文说明
`UNAUTHENTICATED`	401	没登录或 token 过期（前端会自动跳登录页）
`NOT_FOUND`	404	资源不存在**或不属于你**（故意不区分，避免泄露 id 是否存在）
`CONVERSATION_NOT_FOUND`	400	指定的会话不存在或不属于你
`AGENT_EXECUTION_FAILED`	502	Agent 执行失败
`MODEL_PROVIDER_ERROR`	502	大模型调用失败（Key 不对 / 余额不足 / 网络不通）
`STRUCTURED_OUTPUT_ERROR`	502	模型没按格式返回文章
`AGENT_LIMIT_EXCEEDED`	429	超过调用限制

九、两个核心工具

`web_search` —— 联网搜索

- 输入关键词，返回标题 + 链接 + 摘要
- **每一条结果都会自动登记进「来源登记簿」**，这是后面防伪造的基础
- 搜索服务挂了不会抛异常，而是返回错误码，Agent 会基于现有信息继续写作（场景 E）
- 结果条数被双重夹紧：`max(1, min(请求值, MAX_SEARCH_RESULTS, 10))`

`read_webpage` —— 读取网页正文

处理顺序（每一步都有原因）：

```
URL 安全校验       → 防 SSRF，只允许 http/https，禁内网
发起请求           → 带超时、带 UA
检查 Content-Type → 只读文本，不下载 exe/zip/图片
限制响应体大小     → 边下边数，超过 2MB 就断开
提取正文           → trafilatura 为主，BeautifulSoup 兜底
清理噪音           → 去掉导航栏、脚本、菜单
长度截断           → 最多 2 万字符，防止撑爆模型上下文
登记来源           → 标记为「读过全文」
```

绝不把原始 HTML 丢给模型，也绝不把网页全文存进数据库。

十、安全设计

这一节是这个项目最重要的部分。

1. 密钥永远不出后端

- 全项目**只有** `apps/agent/services/model_factory.py` **一个文件**允许读 `LLM_API_KEY`
- 其他任何地方都只能 `model = get_chat_model()`
- 前端**不知道任何密钥**，也不直接访问任何大模型 API
- `.env` 被 `.gitignore` 忽略；`ToolCall` 记录里如果出现 `sk-` 会被自动打码

2. 防 AI 编造来源（本项目最有价值的设计）

问题：模型很擅长编一个「看起来很真」的链接，比如 `https://fake-example-123456.com/article`。

解法：**SourceRegistry（来源登记簿）**

```
工具每搜到一个结果、每读成功一个页面
        → 登记进本次运行的登记簿（只记 URL，不记全文）

文章生成完
        → 逐条核对 draft.sources 里的 URL
        → 登记簿里没有的，直接删除
        → 只要删过，就加一条风险提示告诉用户
        → 最终只有校验通过的来源才会写进 Article.source_ids
```

实测效果（真实链路验证）：

```
模型返回了 1 个伪造来源
→ 接口返回 risk_notes:
   ["模型返回了 1 个未经检索验证的来源，这些来源已被系统移除，相关引用请谨慎对待。"]
→ 文章的 sources = []
```

而且：上一次运行搜到的 URL，在这一次运行时不算数，必须重新检索到才有效。

3. 防 SSRF（让别人的服务器去访问我们内网）

`read_webpage` 是唯一会「主动访问外部地址」的地方，也是最大的风险点。三道防线：

1. **协议白名单** —— 只允许 `http` / `https`。`file://`、`ftp://`、`javascript:` 全部拒绝
2. **域名黑名单** —— `localhost`、`*.internal`、`*.local`、`metadata.google.internal` 等
3. **解析后再查 IP** —— ★ 光看域名是防不住的。一个叫 `evil-looking-domain.com` 的域名 完全可以在 DNS 里解析到 `192.168.1.1`。所以必须在**真正解析出 IP 之后**再检查一遍， 私有地址（10./172.16./192.168./127./169.254.）一律拒绝

重定向也必须逐跳重新校验。 否则攻击者可以用一个正常域名 302 跳转到 `http://169.254.169.254/`（云服务器的密钥接口）来绕过前面所有检查。 所以 `read_webpage` 没有用 httpx 的 `follow_redirects`，而是自己写了重定向循环， **每一次跳转都重新过一遍安全校验**。

4. 前端拿不到敏感信息

- `error_message` 只存中文提示，完整堆栈只进服务器日志
- 全局异常处理器兜底：未知异常一律返回 `INTERNAL_SERVER_ERROR`，不泄露任何细节
- 日志里不记录 API Key、Authorization 头、用户密码

5. 不存第三方内容全文

- `ToolCall.result_preview` 最多 500 字
- `Source.snippet` 最多 500 字
- `read_webpage` 读到的全文**只存在于这一次 Agent 执行的内存里**，用完即弃

6. 防越权

所有查询强制带 `user=request.user`。访问别人的文章返回 **404 而不是 403** —— 连「这个 id 存不存在」都不告诉对方。

7. 防 XSS

模型生成的 Markdown 渲染成 HTML 后，用 `nh3` 清洗一遍， `<script>`、`onclick` 之类全部去掉，外链自动加 `rel="noopener noreferrer"`。 前端再用 DOMPurify 洗第二遍（双保险）。

8. 绝不自动发布

`create_article_from_draft` 里写死了 `status=Article.Status.DRAFT`。 即使你说「帮我写文章并直接发布」，V1 也只会生成草稿然后回一句 **「文章已生成草稿，等待用户确认」**。这条是产品红线，不是配置项。

十一、跑测试

```
cd backend
.venv/Scripts/python.exe -m pytest        # Windows
# .venv/bin/python -m pytest              # macOS / Linux
```

测试**完全离线**，不需要 API Key，不花钱，大约 4 秒跑完。

```
138 passed
```

覆盖内容：

文件	数量	测什么
test_url_safety.py	36	SSRF 防护：各种协议、内网 IP、DNS 解析后仍是内网、IPv6 绕过
test_tools.py	25	两个工具的成功/失败/超时/体积限制/重定向到内网/正文提取
test_source_registry.py	22	防伪造来源、URL 归一化、去重
test_agent_service.py	15	**端到端**：真实 LangChain 循环 + 假模型 + 假搜索
test_api.py	40	接口、参数边界、越权访问、错误信封

怎么做到不花钱的？ 见 apps/agent/tests/fakes.py： 用 ScriptedChatModel 按剧本回放模型的回答，但**工具调用、中间件、来源校验、 数据库写入全部是真的**。所以测出来的可信度远高于「把所有东西都 mock 掉」。

而且有一个安全网：**任何测试如果忘了打桩，会立刻失败而不是偷偷去调付费大模型。**

其他检查：

```
python manage.py check                    # 配置检查
python manage.py makemigrations --check # 确认模型和迁移文件一致
cd ../frontend && npm run build          # 前端类型检查 + 打包
```

十二、常见问题

Q：生成很慢，一直转圈？ A：正常。Agent 要真的联网搜索、打开网页、读完再写，30~120 秒是常态。 前端这个接口的超时单独设成了 5 分钟。

Q：报错「还没有配置 LLM_MODEL，请在后端 .env 文件里填写模型名称」 A：.env 里的 LLM_MODEL 是空的。填上模型名，比如 gpt-4o-mini、deepseek-chat。

Q：报错「调用大模型失败」 A：依次检查 `LLM_API_KEY` 对不对、`LLM_MODEL` 名字对不对、 `LLM_BASE_URL` 需不需要填（用官方 OpenAI 就留空）、账户余额够不够。

Q：生成的文章里说「未能获取到任何外部资料」 A：`SEARCH_API_KEY` 没配。去 https://tavily.com 注册一个免费账号填上就好了。 不配也能用，但它只能靠模型已有知识写，时效性没保障 —— 所以系统会如实提示你。

Q：MySQL 连不上 / 报版本错误 A：Django 5.2 要求 MySQL 8.0.11+。本机是 5.7 的话， 先把 `DB_ENGINE` 改成 `sqlite` 跑起来，再考虑升级数据库。

Q：端口 8000 被占用 A：换个端口 `python manage.py runserver 8001`， 同时把 `frontend/.env` 里的 `VITE_API_BASE_URL` 改成 `http://localhost:8001/api/v1`。

Q：前端提示「无法连接后端服务」 A：后端没启动，或者端口对不上（见上一条），或者 `.env` 里的 `CORS_ALLOWED_ORIGINS` 没包含前端的地址。

Q：Agent 只搜索不读网页？ A：这是模型自己的判断。如果搜索结果里的摘要已经足够，它可能就不打开原文了 —— 这本身是合理行为，System Prompt 里也是这么教它的。想让它读得更勤， 可以在「补充要求」里写「请打开原文核实具体数据」。

十三、V1 没做的事

按需求文档约定，以下**明确不在 V1 范围内**：

- ❌ 多 Agent 协作、MCP、RAG 向量库
- ❌ Redis / Celery / 微服务 / K8s（生成是同步的，链路最短最好调试）
- ❌ **自动发布到微信公众号**（这需要公众号的开发者权限，且风险太高，永远应该人工确认）
- ❌ AI 配图 / 视频生成
- ❌ 小红书 / 抖音等多平台分发
- ❌ 复杂权限、团队协作、付费系统

V1.1 可以考虑的方向（按性价比排序）：

1. **SSE 流式推送** —— 现在 POST 要等 30~120 秒，改成流式能看到「正在搜索...正在读第 2 个网页...」
2. **异步任务化** —— 接 Celery 或 Django 的 background task，请求立刻返回，前端轮询/订阅进度
3. **LangGraph 化** —— 把「大纲人工确认 → 写作 → 事实核查」做成固定流程图，让用户在中途介入
4. **多平台适配** —— 小红书/抖音的文案风格差异很大，可以在 Prompt 层加平台模板
5. **来源可信度评分** —— 按域名给来源打分，优先引用权威来源

许可

仅供学习与内部使用。